

Product Requirements Document (PRD)

CostLens AI — Cloud Cost Intelligence & Root Cause Analysis Platform

Problem statement - **A cloud platform exports infrastructure billing, deployment history, and service usage metrics independently, but finance teams cannot attribute sudden cost spikes to specific engineering activities or releases.**

Field	Detail
Version	1.0.0
Status	Approved – In Development
Created	2026-06-29
Last Updated	2026-06-29
Document Type	Product Requirements Document
Project	CostLens AI
Tech Stack	Python · Pandas · NumPy · SQL · Scikit-learn · Streamlit · Power BI · GitHub

Table of Contents

- 1. Executive Summary
- 2. Business Problem
- 3. User Personas
- 4. User Pain Points
- 5. Project Goals
- 6. Dataset & Data Source Documentation
- 7. Success Metrics
- 8. Functional Requirements
- 9. Non-Functional Requirements
- 10. User Stories
- 11. MVP Scope
- 12. Future Scope
- 13. Risks and Assumptions

1. Executive Summary

CostLens AI is a cloud cost intelligence platform that transforms disconnected billing, infrastructure, deployment, and team ownership data into actionable financial insights. It helps engineering, finance, and leadership teams move from **reactive billing reviews** to **proactive cost governance**.

The platform answers the three most critical FinOps questions at any organization:

Why did cloud costs increase? Who caused it? What should we do about it?

Built as a data product using Python, SQL, machine learning, and Streamlit, CostLens AI delivers:

- Real-time cost visibility broken down by service, team, and environment
- Automated root cause analysis linking cost spikes to specific deployments
- ML-powered anomaly detection and spend forecasting
- Actionable optimization recommendations for engineering and finance teams

This document defines the full product requirements, user stories, MVP boundaries, and acceptance criteria needed to build, validate, and ship CostLens AI.

2. Business Problem

2.1 Context

Modern cloud platforms (AWS, GCP, Azure) generate massive volumes of billing, usage, and infrastructure data. While this data exists, it is typically:

- **Fragmented** across multiple tools (billing consoles, monitoring dashboards, ticketing systems)
- **Delayed** — billing reports arrive days after spending has already occurred
- **Unattributed** — costs are rarely linked to the team, service, or deployment that caused them
- **Unanalyzed** — raw numbers without context are meaningless for decision-making

2.2 The Core Gap

There is no unified intelligence layer that connects:

Data Source	What it tells us
Cloud Billing Data	How much was spent
Infrastructure Logs	What resources were used
Deployment History	What changed and when
Application Metrics	How traffic behaved
Team Ownership Data	Who owns what

Without connecting these dots, organizations cannot answer **why** costs increased — only **that** they did.

2.3 Business Impact

- Engineering teams waste hours during post-mortems trying to correlate deployments with billing spikes
- Finance teams cannot hold specific teams accountable for overruns
- FinOps practitioners lack predictive signals to prevent budget overruns
- Leadership cannot make data-driven infrastructure investment decisions

2.4 Opportunity

Organizations that implement cloud cost intelligence platforms consistently report 15–30% reduction in cloud waste. CostLens AI aims to make that level of intelligence accessible without requiring enterprise-grade tooling.

3. User Personas

3.1 Persona 1 — The Engineering Lead

Name: Arjun Mehta | **Role:** Senior Engineering Manager

Background: Leads a team of 12 engineers building microservices on GCP. Responsible for the team's infrastructure budget.

Attribute	Detail
Primary Goal	Understand which deployments are causing cost increases so he can act quickly
Frustration	Billing reports arrive 3 days late with no deployment context

Attribute	Detail
Tool Comfort	High — comfortable with dashboards, SQL, and GitHub
Success	Can identify a cost-causing deployment within minutes, not days

3.2 Persona 2 — The FinOps Analyst

Name: Divya Nair | **Role:** Cloud FinOps Analyst

Background: Manages cloud spending across 6 product teams. Spends most of her time reconciling billing reports with infrastructure changes.

Attribute	Detail
Primary Goal	Attribute every dollar of cloud spend to a specific team and service
Frustration	Manual reconciliation across billing exports and Jira tickets takes days
Tool Comfort	Medium — uses Excel, Power BI, and basic SQL
Success	Can generate a cost attribution report per team in under 5 minutes

3.3 Persona 3 — The CTO / VP Engineering

Name: Priya Sharma | **Role:** VP of Engineering

Background: Accountable for the entire engineering budget. Attends monthly business reviews and needs cloud cost summaries.

Attribute	Detail
Primary Goal	High-level cost health view with risk flags and forecasts
Frustration	Receives raw billing exports with no narrative or actionable insight

Attribute	Detail
Tool Comfort	Low on tools — needs clean visual dashboards, not raw data
Success	Gets a 2-minute cloud cost summary before every business review

3.4 Persona 4 — The Data / Platform Engineer

Name: Kiran Raj | **Role:** Data Engineer

Background: Responsible for building and maintaining data pipelines. Cares about data quality, schema consistency, and pipeline reliability.

Attribute	Detail
Primary Goal	Build clean, reliable data ingestion pipelines for cost data
Frustration	Inconsistent schema across raw billing exports causes pipeline failures
Tool Comfort	Very High — Python, SQL, Airflow, dbt
Success	Data pipelines run daily with zero manual intervention and clear error logs

4. User Pain Points

#	Persona	Pain Point	Severity
P1	Engineering Lead	Cannot correlate a deployment with a billing spike without hours of manual investigation	● Critical
P2	FinOps Analyst	Cost attribution to teams requires manual cross-referencing of 3–4 tools	● Critical

#	Persona	Pain Point	Severity
P3	CTO / VP Eng	Monthly billing reports have no predictive signals or trend analysis	● High
P4	FinOps Analyst	Anomalies are discovered after the billing cycle, not during	● Critical
P5	Engineering Lead	No visibility into which services are approaching budget limits	● High
P6	CTO / VP Eng	Cannot forecast cloud spend for next month or next quarter	● High
P7	Data Engineer	Raw billing exports have inconsistent schemas and missing fields	● Medium
P8	All Personas	No single source of truth for cloud cost data across teams	● Critical
P9	FinOps Analyst	Optimization recommendations are generic — not tied to actual usage patterns	● Medium
P10	Engineering Lead	Cannot quickly identify if a cost spike is from user traffic growth or infrastructure waste	● High

5. Project Goals

5.1 Primary Goals

Goal	Description
G1 — Cost Visibility	Provide a unified view of cloud spending across services, teams, environments, and time periods
G2 — Root Cause Analysis	Link cost increases to specific deployments, services, or infrastructure changes
G3 — Anomaly Detection	Automatically detect and alert on unusual spending patterns
G4 — Predictive Forecasting	Forecast weekly and monthly cloud spend using ML models
G5 — Cost Attribution	Attribute every dollar of spending to a team, service, and project
G6 — Optimization Insights	Surface specific, data-backed optimization recommendations

5.2 Secondary Goals

Goal	Description
G7 — Executive Reporting	Generate business-friendly dashboards and exportable reports
G8 — Data Product Foundation	Build a reusable, modular data product architecture for future extension
G9 — FinOps Culture	Enable a culture of cost accountability across engineering teams

6. Dataset & Data Source Documentation

The following datasets are required to build **CostLens AI**. Each dataset is documented with its source, ownership, key fields, validation requirements, and refresh frequency to ensure reliable data processing and analytics.

Data Source	Owner	Key Fields	Data Quality / Validation	Refresh Rate
Cloud Billing Data	Cloud Platform / FinOps Team	billing_date, service_name, project_id, cost_usd	Schema validated before ingestion; missing or malformed records are logged	Daily
Infrastructure Usage Data	Platform Engineering Team	cpu_utilization, memory_gb, storage_gb, requests	Missing values handled during preprocessing; schema consistency verified	Daily
Deployment History	DevOps Team	deployment_id, service_name, version, deployment_time	Duplicate records removed; timestamps validated for consistency	Daily
Team Ownership Data	Engineering Management	service_name, team_name, cost_center	Ownership mappings verified before processing	On Update

Data Validation Requirements

- All datasets must pass schema validation before ingestion.
 - Missing or malformed records must be identified and logged without interrupting the pipeline.
 - Duplicate records must be detected and removed during preprocessing.
 - All processed datasets must conform to the defined schema before analytics and machine learning workflows are executed.
-

7. Success Metrics

7.1 Product Metrics (KPIs)

Metric	Target	Measurement Method
Time-to-Root-Cause	< 5 minutes from spike detection to causal deployment	User testing & session recording
Cost Attribution Coverage	100% of spending attributed to a team + service	Data completeness check
Anomaly Detection Precision	≥ 85% precision on labeled test anomalies	ML model evaluation
Forecast Accuracy (MAPE)	≤ 10% mean absolute percentage error on 7-day forecast	Held-out test set evaluation
Dashboard Load Time	< 3 seconds on standard hardware	Performance testing
Data Freshness	Daily pipeline execution with < 1% data loss	Pipeline monitoring

7.2 Business Metrics

Metric	Target
Reduction in manual cost reconciliation time	≥ 60% reduction vs. baseline
% of spending with clear team ownership	≥ 95%
Budget overrun incidents caught before month-end	≥ 80%

7.3 Technical Metrics

Metric	Target
Pipeline test coverage	≥ 80%
Data schema validation pass rate	100% on processed datasets
GitHub Actions CI pipeline success rate	≥ 95%

8. Functional Requirements

8.1 Data Ingestion Module

ID	Requirement	Priority
FR-01	The system SHALL ingest cloud billing data in CSV/JSON format	Must Have
FR-02	The system SHALL ingest infrastructure usage metrics (CPU, memory, storage)	Must Have
FR-03	The system SHALL ingest deployment event logs with timestamps and service tags	Must Have
FR-04	The system SHALL ingest team-to-service ownership mappings	Must Have
FR-05	The system SHALL validate schema on ingestion and log errors for malformed records	Must Have
FR-06	The system SHALL support both batch and simulated streaming ingestion	Should Have

8.2 Data Processing Module

ID	Requirement	Priority
FR-07	The system SHALL clean and normalize all ingested datasets	Must Have
FR-08	The system SHALL join billing, usage, deployment, and ownership data into a unified fact table	Must Have
FR-09	The system SHALL handle missing values with documented imputation strategies	Must Have

ID	Requirement	Priority
FR-10	The system SHALL detect and remove duplicate records	Must Have
FR-11	The system SHALL produce a processed dataset ready for analytics and ML	Must Have

8.3 Analytics & KPI Engine

ID	Requirement	Priority
FR-12	The system SHALL compute daily, weekly, and monthly cost totals by service	Must Have
FR-13	The system SHALL compute cost per team and cost per deployment	Must Have
FR-14	The system SHALL compute week-over-week and month-over-month cost delta	Must Have
FR-15	The system SHALL surface the top 5 most expensive services by period	Must Have
FR-16	The system SHALL compute a Cost Health Score (0–100) per team	Should Have
FR-17	The system SHALL compute budget utilization percentage per team	Should Have
FR-18	The system SHALL generate SQL-powered cost attribution reports	Must Have

8.4 Machine Learning Module

ID	Requirement		Priority
FR-19	The system SHALL train a cost forecasting model to predict 7-day and 30-day spend		Must Have
FR-20	The system SHALL train an anomaly detection model to flag unusual spending events		Must Have
FR-21	The system SHALL label detected anomalies with the associated service and time window		Must Have
FR-22	The system SHALL output forecast confidence intervals alongside point estimates		Should Have
FR-23	The system SHALL retrain models when new data is available (batch retraining)		Should Have
FR-24	The system SHALL store model performance metrics (MAPE, F1, precision, recall)		Must Have

8.5 Dashboard Module (Streamlit)

ID	Requirement	Priority
FR-25	The dashboard SHALL render an Executive View with total spend, forecast, and budget risk	Must Have

ID	Requirement	Priority
FR-26	The dashboard SHALL render an Engineering View with cost by service, team, and deployment	Must Have
FR-27	The dashboard SHALL render a FinOps View with anomalies, attribution, and recommendations	Must Have
FR-28	The dashboard SHALL render an AI Insights panel with natural language cost summaries	Should Have
FR-29	The dashboard SHALL support date range filtering across all views	Must Have
FR-30	The dashboard SHALL support team/service filtering	Must Have
FR-31	The dashboard SHALL allow CSV export of any displayed report	Should Have
FR-32	The dashboard SHALL display anomalies visually highlighted on time-series charts	Must Have

8.6 Reporting Module

ID	Requirement	Priority
FR-33	The system SHALL generate a weekly cost summary report in markdown/PDF format	Should Have
FR-34	The system SHALL generate a team cost ranking report	Should Have

ID	Requirement	Priority
FR-35	The system SHALL produce a Power BI-compatible data export	Should Have

9. Non-Functional Requirements

9.1 Performance

ID	Requirement
NFR-01	Dashboard must load within 3 seconds on a standard laptop with sample datasets
NFR-02	Data processing pipeline must complete within 5 minutes for 1 year of daily records
NFR-03	ML model inference must return predictions within 2 seconds

9.2 Reliability

ID	Requirement
NFR-04	Ingestion pipeline must handle malformed records gracefully without crashing
NFR-05	All pipeline failures must be logged with a descriptive error message
NFR-06	The system must maintain 99% uptime during demo and evaluation periods

9.3 Maintainability

ID	Requirement
NFR-07	All modules must follow PEP 8 coding standards
NFR-08	All functions must have docstrings and inline comments

ID	Requirement
NFR-09	Unit tests must achieve $\geq 80\%$ code coverage
NFR-10	GitHub Actions CI must run on every push to main or develop

9.4 Portability

ID	Requirement
NFR-11	The system must run on any machine with Python 3.10+ and dependencies from requirements.txt
NFR-12	All data paths must be configurable via environment variables or a config file

9.5 Security

ID	Requirement
NFR-13	No real credentials or PII should be stored in the repository
NFR-14	All sensitive configuration must use .env files excluded from version control

9.6 Usability

ID	Requirement
NFR-15	The dashboard must be navigable by a non-technical user without training
NFR-16	All charts must include axis labels, titles, and tooltips

10. User Stories

Epic 1 — Cost Visibility

US-101

As an Engineering Lead, **I want to** see a breakdown of cloud costs by service for the current month, **So that** I can identify which services are consuming the most budget.

Acceptance Criteria:

- ☐ Dashboard displays a bar chart of cost by service for the selected date range
 - ☐ Top 5 most expensive services are highlighted
 - ☐ User can filter by team or environment
 - ☐ Data is current to the latest ingestion run
-

US-102

As a FinOps Analyst, **I want to** see cloud costs attributed to each team with percentage share, **So that** I can hold teams accountable during budget reviews.

Acceptance Criteria:

- ☐ Cost per team is displayed as a ranked table and pie chart
 - ☐ Each team row shows: Total Spend, % of Total, Week-over-Week Change
 - ☐ Data can be exported as CSV
 - ☐ Attribution covers 100% of spending in the dataset
-

US-103

As a VP Engineering, **I want to** see total cloud spend vs. budget with a forecasted end-of-month projection, **So that** I can assess budget risk before the monthly business review.

Acceptance Criteria:

- ☐ Executive view shows: Actual Spend, Budget, Forecast, Variance
 - ☐ Budget utilization is shown as a percentage progress bar
 - ☐ Risk level is color-coded (Green / Yellow / Red)
 - ☐ Forecast is generated by the ML model with $\leq 10\%$ MAPE
-

Epic 2 — Root Cause Analysis

US-201

As an Engineering Lead, **I want to** see which deployments occurred during or just before a cost spike, **So that** I can quickly identify the likely cause of the increase.

Acceptance Criteria:

- ☐ Cost time-series chart overlays deployment events as vertical markers
 - ☐ Clicking a deployment marker shows: service name, version, deploy time, and cost delta
 - ☐ System highlights deployments within a ± 24 h window of detected anomalies
 - ☐ Root cause summary is displayed as a short text insight
-

US-202

As a FinOps Analyst, **I want to** understand whether a cost spike was caused by user traffic growth or infrastructure waste, **So that** I can recommend the right corrective action.

Acceptance Criteria:

- ☐ System correlates cost change with request volume change for the same period
 - ☐ Output distinguishes between: Traffic-Driven Growth vs. Efficiency Degradation
 - ☐ Insight panel shows: cost per request trend over the period
 - ☐ Recommendations differ based on root cause classification
-

US-203

As an Engineering Lead, **I want to** see the cost impact of each deployment across services, **So that** I can understand which releases are the most expensive.

Acceptance Criteria:

- ☐ Deployment cost impact table shows: Deployment ID, Service, Deploy Date, Cost Before, Cost After, Delta
 - ☐ Deployments are ranked by cost delta (highest first)
 - ☐ Table is filterable by team and time range
-

Epic 3 — Anomaly Detection

US-301

As a FinOps Analyst, **I want to** receive automatic alerts when a service's daily spend deviates significantly from its baseline, **So that** I can investigate before the anomaly grows into a budget overrun.

Acceptance Criteria:

- ☐ Anomaly detection model runs on each daily data refresh
 - ☐ Anomalies are flagged with: Service Name, Date, Expected Spend, Actual Spend, Deviation %
 - ☐ Anomalies are visually highlighted on the cost time-series chart
 - ☐ Precision of anomaly detection $\geq 85\%$ on labeled test data
-

US-302

As a VP Engineering, **I want to** see a summary of all active anomalies on the executive dashboard, **So that** I can escalate to the right team if necessary.

Acceptance Criteria:

- ☐ Executive view shows active anomaly count with severity badges
 - ☐ Clicking an anomaly routes to the detailed FinOps view
 - ☐ Anomalies older than 7 days without resolution are marked as persistent
-

Epic 4 — Predictive Forecasting

US-401

As a FinOps Analyst, **I want to** see a 7-day and 30-day cost forecast per service, **So that** I can proactively flag services at risk of budget overrun.

Acceptance Criteria:

- ☐ Forecast chart displays historical spend and projected spend on a single timeline
 - ☐ Confidence interval bands are shown around the point forecast
 - ☐ Forecast is available per-service and for total cloud spend
 - ☐ MAPE on the test set is $\leq 10\%$
-

US-402

As a VP Engineering, **I want to** see the probability of a budget overrun for the current month, **So that** I can make a go/no-go decision on new infrastructure deployments.

Acceptance Criteria:

- ☐ System computes overrun probability based on forecast vs. remaining budget
 - ☐ Executive dashboard displays: "X% chance of overrun by end of month"
 - ☐ Risk is refreshed on every data pipeline run
-

Epic 5 — Optimization Recommendations

US-501

As an Engineering Lead, **I want to** receive specific, data-backed recommendations to reduce costs, **So that** I can take action rather than just seeing the problem.

Acceptance Criteria:

- ☐ Recommendations are based on actual usage patterns, not generic best practices
 - ☐ Each recommendation shows: Issue, Service, Estimated Monthly Savings, Effort Level
 - ☐ At least 3 recommendations are surfaced per dashboard load
 - ☐ Recommendations are ranked by estimated savings (highest first)
-

US-502

As a FinOps Analyst, **I want to** identify over-provisioned services based on usage vs. allocated resources, **So that** I can recommend right-sizing to engineering teams.

Acceptance Criteria:

- ☐ System computes resource utilization rate per service (usage / allocated)
 - ☐ Services with utilization < 40% are flagged as over-provisioned
 - ☐ Right-sizing recommendation shows: current allocation, suggested allocation, estimated savings
-

Epic 6 — Data Pipeline

US-601

As a Data Engineer, **I want to** run the full data ingestion and processing pipeline with a single command, **So that** I can refresh data without manual intervention.

Acceptance Criteria:

- ☐ Pipeline is executable via `python src/ingestion/run_pipeline.py`
 - ☐ Pipeline validates schema, cleans data, and writes to the processed layer
 - ☐ Pipeline logs success/failure with timestamps and record counts
 - ☐ Malformed records are written to a separate error log, not dropped silently
-

US-602

As a Data Engineer, **I want to** have CI checks run automatically on every code push, **So that** I can catch breaking changes before they reach the main branch.

Acceptance Criteria:

- ☐ GitHub Actions workflow runs on push to main and develop
 - ☐ CI includes: linting (flake8), unit tests (pytest), and schema validation
 - ☐ Failing CI blocks merge to main
 - ☐ CI badge is visible on the README
-

11. MVP Scope

The MVP focuses on validating the core value proposition: **connect billing data to deployments and surface root causes automatically.**

 Included in MVP

Feature	Rationale
Data ingestion from CSV (billing, usage, deployments, ownership)	Foundation for everything else
Data cleaning and unified fact table creation	Required for analytics and ML
Cost breakdown by service, team, and period (SQL-powered)	Core value — cost visibility
Deployment overlay on cost time-series	Core value — root cause

Feature	Rationale
Isolation Forest anomaly detection	Core value — proactive alerting
Linear Regression / Random Forest cost forecasting	Core value — predictive signals
Streamlit dashboard with Executive, Engineering, FinOps views	Primary delivery surface
GitHub Actions CI pipeline	Quality gate
Sample/simulated datasets	Enables demo without real cloud data

✗ Explicitly Excluded from MVP

Feature	Reason for Deferral
Real-time streaming ingestion	Adds infrastructure complexity; not needed for MVP validation
LLM-powered natural language Q&A	Requires API cost and additional engineering; Phase 2
Multi-cloud support (AWS, Azure)	Schema complexity; MVP targets single cloud (GCP/simulated)
Slack/Email anomaly alerts	External integrations; Phase 2
Power BI live integration	Requires Power BI workspace; export to CSV suffices for MVP
User authentication on dashboard	Not required for internal project evaluation

12. Future Scope

Phase 2 — Intelligence Layer

Feature	Description
AI Cost Advisor	Natural language Q&A interface powered by an LLM (e.g., Gemini, GPT-4) to answer questions about cost data
LLM Executive Summaries	Auto-generate narrative reports from cost data for leadership distribution

Feature	Description
Slack / Email Anomaly Alerts	Push real-time anomaly notifications to team channels
Advanced Forecasting	XGBoost and LSTM-based models for higher accuracy predictions

Phase 3 — Scale & Integration

Feature	Description
Multi-Cloud Support	Ingest and normalize billing data from AWS Cost Explorer and Azure Cost Management
Real-Time Streaming	Apache Kafka or Pub/Sub based streaming ingestion for near-real-time dashboards
RBAC on Dashboard	Role-based access control — engineers see their team's data, FinOps sees all
Automated Optimization Actions	Trigger infrastructure changes (e.g., scale-down recommendations via Terraform)
dbt Integration	Replace raw SQL with dbt models for modular, tested transformation logic

Phase 4 — Enterprise Readiness

Feature	Description
SaaS Deployment	Containerized deployment on GCP Cloud Run or AWS Fargate
API Layer	REST API for programmatic access to cost intelligence data
Audit Trail	Log all cost attribution decisions for compliance and audit purposes
Custom Budget Rules	Allow teams to define their own alert thresholds and budget limits

13. Risks and Assumptions

13.1 Risks

#	Risk	Probability	Impact	Mitigation
R1	Simulated data may not reflect real-world billing complexity	Medium	High	Design data schemas based on actual GCP/AWS billing export formats
R2	ML models may underperform on small or synthetic datasets	High	Medium	Use cross-validation; document MAPE clearly; set realistic accuracy targets
R3	Streamlit performance may degrade with large datasets	Medium	Medium	Implement data caching (@st.cache_data) and pagination
R4	Schema inconsistencies across simulated datasets	Medium	High	Define and enforce a strict schema contract in src/ingestion/schema.py
R5	GitHub Actions CI may fail due to environment inconsistencies	Low	Medium	Pin all dependency versions in requirements.txt ; test CI locally first
R6	Scope creep beyond MVP boundaries	High	High	Strict adherence to MVP scope; defer all Phase 2+ features

#	Risk	Probability	Impact	Mitigation
R7	Deployment-to-cost correlation may be weak in synthetic data	Medium	High	Explicitly engineer the correlation in data generation scripts

13.2 Assumptions

#	Assumption
A1	Data is available in CSV format; no live cloud API connection is required for MVP
A2	Simulated datasets can adequately represent real-world cloud billing patterns for MVP validation
A3	Streamlit is the accepted primary dashboard interface; Power BI is secondary (export-based)
A4	Python 3.10+ is available in the development environment
A5	GitHub is the version control and CI/CD platform
A6	All team members are familiar with Python, Pandas, and basic SQL
A7	The project will be evaluated in a demo environment, not a production cloud setting

14. Acceptance Criteria

The following criteria define the minimum bar for the CostLens AI MVP to be considered complete and ready for evaluation.

AC-1 — Data Pipeline

- ☐ All four datasets (billing, usage, deployments, ownership) are successfully ingested
- ☐ Processed fact table is produced with zero schema violations
- ☐ Malformed records are isolated and logged without crashing the pipeline
- ☐ Pipeline completes in under 5 minutes for 1 year of daily data

AC-2 — Analytics Engine

- ☐ Cost totals are accurate to within 0.01% of expected values (verified against source data)
- ☐ Cost attribution covers 100% of records in the processed dataset
- ☐ Week-over-week and month-over-month deltas are computed for all services

AC-3 — Machine Learning

- ☐ Anomaly detection model achieves $\geq 85\%$ precision on the labeled test set
- ☐ Cost forecasting model achieves $\leq 10\%$ MAPE on the held-out test set
- ☐ Model training and inference scripts run without errors end-to-end
- ☐ Model artifacts are saved and reloadable without retraining

AC-4 — Dashboard

- ☐ All three views (Executive, Engineering, FinOps) render without errors
- ☐ Date range and team/service filters work correctly across all views
- ☐ Anomalies are visually highlighted on the time-series chart
- ☐ Deployment events are overlaid on the cost chart
- ☐ Dashboard loads in under 3 seconds on a standard laptop

AC-5 — Root Cause Analysis

- ☐ For any selected anomaly, the system displays the associated deployment within $\pm 24\text{h}$
- ☐ The system distinguishes traffic-driven cost increases from efficiency degradation
- ☐ A human-readable root cause summary is displayed in the Insights panel

AC-6 — Code Quality & DevOps

- ☐ All Python files pass flake8 linting with no errors
- ☐ Unit test coverage $\geq 80\%$ as measured by `pytest --cov`
- ☐ GitHub Actions CI runs successfully on the main branch
- ☐ README.md includes setup instructions, architecture overview, and screenshot

AC-7 — Documentation

- ☐ PRD is complete and version-controlled in docs/PRD.md
 - ☐ All source files have module-level docstrings
 - ☐ requirements.txt is accurate and reproducible
-

Appendix A — Glossary

Term	Definition
FinOps	Financial Operations — the practice of bringing financial accountability to cloud spending
Root Cause Analysis	The process of identifying the underlying reason for a cost increase or anomaly
Anomaly	A data point that deviates significantly from expected patterns
MAPE	Mean Absolute Percentage Error — a metric for forecast accuracy
Isolation Forest	An unsupervised ML algorithm for outlier/anomaly detection
Cost Health Score	A composite 0–100 score representing a team's cost efficiency
Attribution	The act of assigning a portion of total cloud spending to a specific team, service, or deployment
Fact Table	A central database table in a star schema containing measurable, quantitative data

Appendix B — Data Schema Contracts

Billing Dataset

None		
date	:	DATE (YYYY-MM-DD)
service_name	:	VARCHAR(100)
environment	:	VARCHAR(50) (prod staging dev)
cost_usd	:	DECIMAL(10,2)
currency	:	VARCHAR(10)
project_id	:	VARCHAR(50)

Deployment Dataset

None

```
deployment_id : VARCHAR(50)
date          : DATETIME
service_name  : VARCHAR(100)
version       : VARCHAR(20)
deployed_by   : VARCHAR(100)
environment   : VARCHAR(50)
status        : VARCHAR(20) (success | failed | rollback)
```

Usage / Infrastructure Dataset

None

```
date          : DATE
service_name  : VARCHAR(100)
cpu_utilization : DECIMAL(5,2) (%)
memory_gb     : DECIMAL(6,2)
requests      : INTEGER
storage_gb    : DECIMAL(8,2)
```

Team Ownership Dataset

None

```
service_name : VARCHAR(100)
team_name    : VARCHAR(100)
team_lead    : VARCHAR(100)
cost_center  : VARCHAR(50)
```
